



University of Applied Sciences  
Fakultät für Informatik und Wirtschaftsinformatik

---

# Raytracer-Optimierung

Whitted-Style Raytracer mit Badouel & Möller-Trumbore

## Projektbericht

Konzepte von Programmiersprachen  
Sommersemester 2026

---

**Betreuer:** Prof. Dr. Christian Pape

**Autor:** Erlind Sejdiu

**Abgabe:** 29. April 2026

## Inhaltsverzeichnis

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Einleitung</b>                                               | <b>2</b> |
| <b>2</b> | <b>Aufgabe 1: Whitted-Style Raytracer mit Brute-Force-Suche</b> | <b>2</b> |
| 2.1      | Architektur und Modulstruktur . . . . .                         | 2        |
| 2.2      | Badouel-Algorithmus (Baseline) . . . . .                        | 4        |
| 2.3      | Möller-Trumbore-Algorithmus (1. Optimierung) . . . . .          | 4        |
| 2.4      | Brute-Force-Schnittpunktsuche . . . . .                         | 5        |
| 2.5      | Beleuchtung und Schatten . . . . .                              | 5        |
| 2.6      | Zeitmessung . . . . .                                           | 6        |
| <b>3</b> | <b>Aufgabe 2: Beschleunigungsdatenstruktur</b>                  | <b>7</b> |
| 3.1      | Struktur und Aufbau des k-d-Baums . . . . .                     | 7        |
| 3.2      | Traversierung und Slab-Test . . . . .                           | 7        |
| 3.3      | Optional: Surface-Area-Heuristik (SAH) . . . . .                | 7        |
| <b>4</b> | <b>Aufgabe 3: AVX-Optimierung mit Packet Tracing</b>            | <b>8</b> |
| <b>5</b> | <b>Zusammenfassung und Ausblick</b>                             | <b>8</b> |

## 1 Einleitung

Gegenstand dieses Projektberichts ist die schrittweise Implementierung und Optimierung eines einfachen Raytracers im Rahmen der Veranstaltung *Konzepte von Programmiersprachen*. Der Raytracer wurde in C++17 unter Verwendung der Mathematikbibliothek GLM entwickelt und mit dem GCC-Compiler über CMake übersetzt. Als Testszene dient die bereitgestellte Wavefront-Datei `teapot_n_glass.obj`, die eine Teekanne mit Glasobjekt umfasst und insgesamt 18 032 Dreiecke enthält. Alle Zeitmessungen beziehen sich auf diese Szene bei einer festen Auflösung von  $500 \times 500$  Bildpunkten, um über die Teilaufgaben hinweg vergleichbare Ergebnisse zu erzielen.

Der Bericht dokumentiert die Ergebnisse aller Teilaufgaben. Jeder Abschnitt beschreibt die gewählte Vorgehensweise, die wesentlichen Implementierungsdetails sowie die gemessenen Renderzeiten.

## 2 Aufgabe 1: Whitted-Style Raytracer mit Brute-Force-Suche

### 2.1 Architektur und Modulstruktur

Das Programm ist modular aufgebaut und in mehrere logisch getrennte Einheiten unterteilt. Die folgende Tabelle gibt einen Überblick über alle Dateien des Projekts:

Tabelle 1: Übersicht der Projektdateien

| Datei                           | Typ      | Beschreibung                                                                                                                                                                |
|---------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ray.h</code>              | Header   | Definiert die Datenstruktur <code>Ray</code> (Ursprung + Richtungsvektor).                                                                                                  |
| <code>camera.h</code>           | Header   | Kameralogik: Berechnet aus Position, Blickpunkt und FOV das lokale Koordinatensystem und erzeugt Primärstrahlen.                                                            |
| <code>mesh.h/.cpp</code>        | Modul    | Einlesen der Szenengeometrie aus Wavefront-OBJ-Dateien inkl. Materialien (MTL). Enthält die <code>Triangle</code> -Struktur und eine Methode zum Einfügen einer Bodenebene. |
| <code>intersector.h/.cpp</code> | Modul    | Schnittpunktalgorithmen: Möller-Trumbore und Badouel. Definiert auch die <code>Intersection</code> -Struktur und das <code>IntersectionAlgorithm</code> -Enum.              |
| <code>shading.h/.cpp</code>     | Modul    | Beleuchtungsberechnung: Blinn-Phong-Modell (Ambient + Diffuse + Specular) und Shadow-Ray-Logik. Enthält die <code>Light</code> -Struktur.                                   |
| <code>renderer.h/.cpp</code>    | Modul    | Render-Loop mit Anti-Aliasing (Multisampling). Delegiert die Beleuchtung an das Shading-Modul.                                                                              |
| <code>scene.h/.cpp</code>       | Modul    | Szenenkonfiguration: Bündelt Mesh, Kamera und Lichtquelle. Lädt das Modell und fügt die Bodenebene hinzu.                                                                   |
| <code>image.h/.cpp</code>       | Modul    | Bildspeicher und PPM-Export. Verwaltet den Puffer und schreibt das Ergebnis als PPM-Datei.                                                                                  |
| <code>main.cpp</code>           | Einstieg | Programmstart: Erstellt die Szene, führt beide Algorithmen aus und misst die Renderzeiten.                                                                                  |

Das Modul `Ray` definiert die grundlegende Datenstruktur eines Sehstrahls, bestehend aus einem Ursprungspunkt und einem normalisierten Richtungsvektor. Das Modul `Camera` kapselt die Kameralogik und berechnet aus einer gegebenen Position, einem Blickpunkt, einem Up-Vektor sowie einem vertikalen Öffnungswinkel das lokale Koordinatensystem der Kamera. Für jeden Bildpunkt kann daraus ein Primärstrahl erzeugt werden, indem die normalisierten Bildkoordi-

naten  $(u, v)$  auf die Bildebene im Weltkoordinatensystem abgebildet werden.

Das Modul **Mesh** ist für das Einlesen der Szenengeometrie aus dem Wavefront-OBJ-Format zuständig. Es parst die Datei zeilenweise und extrahiert Vertex-Positionen (Zeilen mit Präfix **v**), Vertex-Normalen (**vn**) sowie Dreiecksdefinitionen (Zeilen mit Präfix **f**). Zusätzlich werden Materialdateien (**.mtl**) geparst, um diffuse Farben (**Kd**), spekulare Farben (**Ks**) und Glanzexponenten (**Ns**) den jeweiligen Dreiecken zuzuordnen.

Das Modul **Intersector** enthält die statischen Methoden zur Schnittpunktberechnung, deren Implementierung im folgenden Abschnitt erläutert wird. Das Modul **Shading** kapselt die gesamte Beleuchtungslogik (Blinn-Phong und Shadow Rays), während das Modul **Scene** alle Szenen-Parameter (Geometrie, Kamera, Licht) in einer zentralen Struktur bündelt.

## 2.2 Badouel-Algorithmus (Baseline)

Als Baseline für die Schnittpunktberechnung zwischen einem Sehstrahl und einem Dreieck wurde der Algorithmus nach Badouel implementiert. Dieser Algorithmus berechnet zunächst explizit die Ebenengleichung des Dreiecks und den potenziellen 3D-Schnittpunkt des Strahls mit dieser Ebene.

Um den anschließenden *Point-in-Polygon*-Test durchzuführen, projiziert das Verfahren den 3D-Schnittpunkt und die Dreieckseckpunkte auf eine 2D-Ebene. Dies geschieht durch das simple Weglassen derjenigen Koordinatenachse, bei der die Flächennormale ihre betragsmäßig größte Ausdehnung besitzt (die sogenannte dominierende Achse). Durch diese Projektion auf die  $xy$ -,  $xz$ - oder  $yz$ -Ebene reduziert sich der Test auf das Lösen eines 2D-Gleichungssystems zur Bestimmung der baryzentrischen Koordinaten. Obwohl dieser Ansatz durch die Reduktion auf zwei Dimensionen prinzipiell Multiplikationen einspart, bildet er aufgrund des Branching-Overheads (zur Bestimmung der dominierenden Achse) sowie der initialen Ebenenberechnungen unsere Performance-Baseline.

## 2.3 Möller-Trumbore-Algorithmus (1. Optimierung)

Als erste Optimierungsmaßnahme gegenüber der Badouel-Baseline wurde zusätzlich der Möller-Trumbore-Algorithmus implementiert. Dieser bietet gegenüber klassischen Verfahren einen entscheidenden Effizienzvorteil: Er berechnet die baryzentrischen Koordinaten  $(u, v)$  direkt im 3D-Raum und vermeidet so die explizite Konstruktion der Dreiecksebene sowie die teuren Fallunterscheidungen für die 2D-Projektion.

Der Algorithmus beruht auf der Umformung der Strahlgleichung  $\mathbf{O} + t \cdot \mathbf{D} = (1 - u - v) \cdot \mathbf{V}_0 + u \cdot \mathbf{V}_1 + v \cdot \mathbf{V}_2$  in ein lineares Gleichungssystem, das mit der Cramerschen Regel gelöst wird. Dabei werden die Kanten  $\mathbf{e}_1 = \mathbf{V}_1 - \mathbf{V}_0$  und  $\mathbf{e}_2 = \mathbf{V}_2 - \mathbf{V}_0$  berechnet, und über Kreuzprodukte und Skalarprodukte die Parameter  $t$ ,  $u$  und  $v$  bestimmt. Ein Treffer liegt genau dann vor, wenn  $u \geq 0$ ,  $v \geq 0$ ,  $u + v \leq 1$  und  $t > \varepsilon$  gilt. Der Schwellwert  $\varepsilon = 10^{-7}$  verhindert numerische Artefakte bei nahezu parallelen Strahlen.

Die zentrale Implementierung stellt sich wie folgt dar:

```

1 Intersection Intersector::intersectRayTriangle(
2     const Ray& ray,
3     const glm::vec3& v0, const glm::vec3& v1, const glm::vec3& v2)
4 {
5     Intersection result = {false, 0.0f, 0.0f, 0.0f};
6
7     glm::vec3 edge1 = v1 - v0;
8     glm::vec3 edge2 = v2 - v0;
9     glm::vec3 h = glm::cross(ray.direction, edge2);
10    float a = glm::dot(edge1, h);
11
12    constexpr float EPSILON = 0.0000001f;
13    if (a > -EPSILON && a < EPSILON)
14        return result;
15
16    float f = 1.0f / a;
17    glm::vec3 s = ray.origin - v0;
18    result.u = f * glm::dot(s, h);
19    if (result.u < 0.0f || result.u > 1.0f)
20        return result;
21
22    glm::vec3 q = glm::cross(s, edge1);
23    result.v = f * glm::dot(ray.direction, q);
24    if (result.v < 0.0f || result.u + result.v > 1.0f)
25        return result;
26
27    result.t = f * glm::dot(edge2, q);
28    if (result.t > EPSILON) {
29        result.hit = true;
30    }
31    return result;
32 }

```

Listing 1: Möller-Trumbore-Schnittpunkttest

Der Algorithmus nutzt ein frühzeitiges Abbruchverfahren (Early Rejection): Sobald einer der baryzentrischen Parameter außerhalb des gültigen Bereichs liegt, wird die Berechnung sofort abgebrochen, ohne die verbleibenden Skalarprodukte auszuwerten. Dies reduziert den Rechenaufwand für die Mehrheit der Strahlen, die ein gegebenes Dreieck verfehlen.

## 2.4 Brute-Force-Schnittpunktsuche

Da in dieser ersten Teilaufgabe noch keine Beschleunigungsdatenstruktur vorliegt, erfolgt die Bestimmung des nächstgelegenen Schnittpunktes durch eine vollständige lineare Suche über alle 18 032 Dreiecke der Szene. Für jeden Primärstrahl wird die `intersectRayTriangle`-Methode auf jedes einzelne Dreieck angewendet und der Treffer mit dem kleinsten  $t$ -Wert gespeichert. Bei einer Bildauflösung von  $500 \times 500$  Pixeln und einem zusätzlichen Schattenstrahl pro beleuchtetem Pixel ergibt sich eine Gesamtzahl von bis zu  $500 \times 500 \times 2 \times 18\,032 = 9,016 \cdot 10^9$  Schnittpunkttests im ungünstigsten Fall.

## 2.5 Beleuchtung und Schatten

Zur Erzeugung eines plastischen Eindrucks wurde ein Blinn-Phong-Beleuchtungsmodell implementiert, das aus ambienter, diffuser und spekularer Komponente besteht. Eine einzelne Punktlichtquelle an der Position  $(6, 8, -5)$  im Weltkoordinatensystem beleuchtet die Szene. Trifft ein Primärstrahl auf ein Dreieck, wird zunächst der exakte Auftreffpunkt  $\mathbf{P} = \mathbf{O} + t \cdot \mathbf{D}$  berechnet. Die Oberflächennormale wird durch baryzentrische Interpolation der Vertex-Normalen bestimmt, was zu einer glatten Schattierung (Smooth Shading) führt.

Um den Schattenwurf sichtbar zu machen, wurde zusätzlich eine Bodenebene (Ground Plane)

unter der Szene eingefügt. Diese besteht aus zwei großen Dreiecken und dient als Empfängerfläche für die Schatten der darüberliegenden Objekte.

Für die Schattenberechnung wird ein sekundärer Strahl (Shadow Ray) vom Auftreffpunkt in Richtung der Lichtquelle ausgesendet. Der Startpunkt wird dabei um einen kleinen Offset in Normalenrichtung (0,001) verschoben, um das Phänomen der Shadow Acne zu vermeiden, bei dem ein Dreieck sich selbst fälschlicherweise verschattet. Trifft der Schattenstrahl auf seinem Weg ein Objekt, dessen Entfernung kleiner ist als die Distanz zur Lichtquelle, wird der Punkt als verschattet deklariert und lediglich mit einem schwachen Umgebungslicht dargestellt. Andernfalls wird die Helligkeit nach dem Lambertschen Beleuchtungsmodell berechnet, bei dem der Kosinus des Einfallswinkels zwischen Lichtrichtung und Oberflächennormale die Intensität bestimmt:

```

1 float distanceToLight = glm::length(lightPos - hitPoint);
2 Ray shadowRay = {hitPoint + normal * 0.001f, lightDir};
3 Intersection shadowIsect;
4 Triangle dummyTriangle;
5 bool inShadow = findClosestHit(shadowRay, scene.triangles,
6                               shadowIsect, dummyTriangle);
7
8 // Nur Objekte ZWISCHEN Trefferpunkt und Lichtquelle erzeugen Schatten
9 if (inShadow && shadowIsect.t > distanceToLight) {
10     inShadow = false;
11 }

```

Listing 2: Schattenstrahl mit Distanzprüfung

Die Distanzprüfung in Zeile 9–11 stellt sicher, dass ausschließlich Objekte, die sich tatsächlich zwischen dem Auftreffpunkt und der Lichtquelle befinden, einen Schatten werfen. Ohne diese Prüfung könnten Objekte hinter der Lichtquelle fälschlicherweise als Okkluder erkannt werden.

## 2.6 Zeitmessung

Die Zeitmessung erfolgt über `std::chrono::high_resolution_clock` und umfasst ausschließlich den Render-Loop ohne die Ladezeit der Szene. Gemessen wurde auf einem System mit Ubuntu unter WSL. Die folgende Tabelle zeigt die Renderzeiten für die Brute-Force-Implementierung ( $500 \times 500$  Pixel, 18 032 Dreiecke).

Tabelle 2: Renderzeiten der Brute-Force-Implementierung ( $500 \times 500$  Pixel, 18 032 Dreiecke)

| Konfiguration                                 | Badouel [ms] | Möller-Trumbore [ms] |
|-----------------------------------------------|--------------|----------------------|
| Release (mit <code>-O3 -march=native</code> ) | 728.592      | 484.488              |

Die Messergebnisse zeigen die Ausführungszeiten im vollständig optimierten Release-Modus (Compiler-Flags `-O3 -march=native`). Die Renderzeiten beinhalten Anti-Aliasing ( $4\times$  Multisampling), Blinn-Phong-Beleuchtung mit Normaleninterpolation, Shadow Rays sowie eine Bodenebene für sichtbare Schatten. Es zeigt sich, dass der Möller-Trumbore-Algorithmus unsere Baseline (Badouel) durch Reduktion von Branching und Verzicht auf Ebenenkonstruktion messbar schlägt. Der Geschwindigkeitsvorteil beträgt hierbei etwa den Faktor  $1,50\times$ .

Trotz des Einsatzes des performanteren Algorithmus bleibt die absolute Renderzeit mit rund 729 Sekunden (Badouel) bzw. 484 Sekunden (Möller-Trumbore) sehr hoch. Die Ursache liegt in der linearen Komplexität  $\mathcal{O}(n)$  der Brute-Force-Suche pro Strahl, wobei  $n$  die Anzahl der Dreiecke in der Szene bezeichnet. Dies motiviert die Einführung einer hierarchischen Beschleunigungsdatenstruktur in der nachfolgenden Aufgabe.

### 3 Aufgabe 2: Beschleunigungsdatenstruktur

#### 3.1 Struktur und Aufbau des k-d-Baums

Um die ineffiziente Brute-Force-Suche abzulösen, wurde als Beschleunigungsdatenstruktur ein k-d-Baum (*k-dimensional tree*) implementiert. Dieser hierarchische Raumunterteilungsbaum partitioniert die 3D-Szene rekursiv in achsenparallele Teilräume (Bounding Boxes).

Der Aufbau des Baums erfolgt *Top-Down*. In jedem Rekursionsschritt wird zunächst eine Axis-Aligned Bounding Box (AABB) berechnet, die alle Dreiecke des aktuellen Knotens umschließt. Anschließend wird die längste Achse dieser Bounding Box als Trennachse bestimmt. Die Dreiecke werden entlang dieser Achse basierend auf ihren Schwerpunkten (Centroids) aufsteigend sortiert. Die Unterteilung erfolgt strikt am **Median**, das heißt, die sortierte Liste wird exakt in zwei gleich große Hälften geteilt, was zu einem gut balancierten Baum führt. Die Rekursion bricht ab, sobald ein Knoten weniger als drei Dreiecke enthält oder die maximale Baumtiefe von 20 erreicht ist. Dieser Knoten wird dann als Blattknoten markiert und referenziert die zugehörigen Dreiecke aus dem umgeschichteten Speicher-Array.

#### 3.2 Traversierung und Slab-Test

Während des Renderns wird der Baum durchquert, um den nächstgelegenen Schnittpunkt eines Strahls mit der Szenengeometrie zu finden. Anstatt jedes Dreieck einzeln zu testen, wird an jedem Knoten ein **Slab-Test** gegen dessen AABB durchgeführt. Der Slab-Test ist sehr effizient umgesetzt und nutzt den vorausberechneten Kehrwert der Strahlrichtung (`invDirection`), wodurch teure Fließkommadivisionen in der Schleife entfallen.

Verfehlt der Strahl die Bounding Box eines Knotens, können alle darunterliegenden Dreiecke sowie Unterbäume komplett übersprungen werden. Ein weiterer entscheidender Optimierungsfaktor in der Implementierung ist die dynamische Begrenzung des Slab-Tests: Als maximal zulässige Distanz (`tMax`) wird die Distanz des bisher nächstgelegenen Treffers (`closestIsect.t`) übergeben. Liegt die Bounding Box eines Knotens weiter entfernt als ein bereits bestätigter Treffer, schlägt der Slab-Test sofort fehl.

Erreicht die Traversierung einen Blattknoten, werden die enthaltenen Dreiecke mittels Möller-Trumbore-Algorithmus (oder Badouel) auf Schnittpunkte geprüft. Durch die Reduzierung der Komplexität von  $\mathcal{O}(n)$  auf den durchschnittlichen Fall von  $\mathcal{O}(\log n)$  verringert sich die Anzahl der zwingend notwendigen Strahl-Dreieck-Tests erheblich. Die Renderzeit der 18 032 Dreiecke fassenden Teekannen-Szene sinkt durch den Einsatz des k-d-Baums drastisch.

#### 3.3 Optional: Surface-Area-Heuristik (SAH)

Aufbauend auf dem bestehenden Beschleunigungsbaum mit Median-Split wurde als weitere Optimierung die Surface-Area-Heuristik (SAH) implementiert. Anstatt den Raum in jedem Schritt stur in der Mitte zu teilen, bewertet SAH verschiedene potenzielle Split-Ebenen anhand der daraus resultierenden Bounding-Box-Oberflächen. Da die Wahrscheinlichkeit, dass ein Strahl eine Bounding-Box trifft, proportional zu ihrer Oberfläche ist, minimiert dieser Ansatz die erwarteten Kosten für zukünftige Schnittpunkttests.

Die Implementierung prüft entlang aller drei Achsen mögliche Split-Positionen, die durch die Schwerpunkte der Dreiecke definiert sind. Durch effiziente Vorabberechnungen der Flächen (mittels Sweeps von links nach rechts und umgekehrt) werden die Kosten in linearer Zeit ermittelt. Der Split mit den geringsten Gesamtkosten wird gewählt, sofern er günstiger ist als die Erstellung eines Blattknotens. Der Wechsel zum SAH-Split hat die Renderzeit von 2931 ms auf 1373 ms weiter drastisch reduziert.



## 4 Aufgabe 3: AVX-Optimierung mit Packet Tracing

Um die Hardware-Ebene der x86-Prozessoren besser auszunutzen, wurde das Tracing der Primärstrahlen mit Hilfe der experimentellen C++ SIMD-Befehle (`<experimental/simd>`) vektorisiert. Hierbei werden kohärente Primärstrahlen (z.B. 8 Strahlen in einer Bildzeile) zu einem `RayPacket` im Structure-of-Arrays (SoA) Format gebündelt.

Dieses Paket wird dann als Einheit durch den Beschleunigungsbaum traversiert. Der Slab-Test in den Bounding-Boxen sowie der Möller-Trumbore-Algorithmus an den Blattknoten werden bis zu 8 Strahlen simultan in Vektorregistern aus. Um Masking-Probleme in den SIMD-Variablen zu umgehen, wird unter anderem der getroffene Dreiecks-Index als `floatv` in der `IntersectionPacket`-Struktur abgelegt. Als ultimative Optimierung wurde zudem die gesamte Schattierungslogik (inkl. Normalenberechnung, Beleuchtung via Punkt- und Kreuzprodukten sowie die Berechnung der Schattenstrahlen) unter Ausnutzung aller `stdx::simd`-Mathematikfunktionen (`stdx::max`, `stdx::pow`, `stdx::clamp` etc.) auf volle Vektorbreite umgestellt. Auch die Schattenstrahlen werden als komplettes Paket parallel auf Okklusion geprüft. Diese tiefgreifende Vektorisierung senkt die Renderzeit auf sagenhafte 403 ms.

## 5 Zusammenfassung und Ausblick

Tabelle 3: Gesamtübersicht der Renderzeiten im Release-Modus (500×500 Pixel)

| Implementierungsschritt          | Renderzeit [ms] | Speedup |
|----------------------------------|-----------------|---------|
| Aufgabe 1: Brute-Force (Badouel) | 728.592         | 1,0×    |
| Aufgabe 1: Brute-Force (M.-T.)   | 484.488         | 1,5×    |
| Aufgabe 2: k-d-Baum (Median)     | 2931            | 248,6×  |
| Aufgabe 2: k-d-Baum (SAH)        | 1373            | 530,7×  |
| Aufgabe 3: AVX Packet Tracing    | 403             | 1807,9× |

Die Entwicklung dieses Raytracers verdeutlicht eindrucksvoll die kumulative Wirkung algorithmischer und technischer Optimierungen. Während die Auswahl des effizienteren Möller-Trumbore-Schnittpunkttests einen soliden Grundbaustein legte (Faktor 1,5), brachte erst die Umstellung der linearen Suchkomplexität auf eine logarithmische Suche durch den k-d-Baum den entscheidenden Durchbruch (Faktor 248).

Die verfeinerte Aufbauheuristik (SAH) verdoppelte diese Performance nochmals, indem sie die Baumstruktur optimal an die Strahlenwahrscheinlichkeiten anpasste. Die abschließende Nutzung der Hardware-Vektorisierung (AVX/SIMD) durch paralleles Packet Tracing sowohl für Primär- als auch Schattenstrahlen sowie vektorisiertes Shading reduzierte die Renderzeit auf 403 Millisekunden – das entspricht einem gigantischen Speedup um den Faktor 1808 gegenüber der ursprünglichen Brute-Force-Variante.



Abbildung 1: Gerendertes Ergebnis der Szene (Teekanne und Glasobjekt) mit Blinn-Phong-Beleuchtung und Schattenwurf auf der Bodenebene